SHERLOCK

Security Review For Venus Protocol

Collaborative Audit Prepared For: **Venus Protocol**
Lead Security Expert(s): **panprog**
TessKimy
Date Audited: **May 21 - May 28, 2026**

Introduction

Venus is the leading decentralized lending protocol on BNB Chain. Founded in 2020 as the first lending protocol on the network, Venus supports over 30 assets and reached $2.8 billion in TVL in 2025.

The protocol offers two products: Venus Core, designed for institutional participants seeking deep liquidity and broad asset coverage, and Venus Flux, a retail-focused product built for enhanced capital efficiency.

Security is foundational to Venus's development practice. Every smart contract change undergoes an independent audit before deployment – resulting in over 80 completed audits across leading firms, among the most extensive audit records in DeFi.

Venus is governed by its community through on-chain DAO governance, with all protocol changes subject to community vote.

Scope

Repository: VenusProtocol/venus-protocol

Audited Commit: 9a6aa2a5f2bdb747606741ca989c6a4de03bf79a

Final Commit: acfafd30aa63c6e126cb3c6b73b4c6a374ea3cba

Files:

- contracts/Tokens/Prime/IPrimeLeaderboard.sol
- contracts/Tokens/Prime/PrimeLeaderboard.sol
- contracts/Tokens/Prime/PrimeLeaderboardStorage.sol
- contracts/Tokens/Prime/PrimeV2.sol
- contracts/Tokens/Prime/PrimeV2Storage.sol

Final Commit Hash

acfafd30aa63c6e126cb3c6b73b4c6a374ea3cba

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.

- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
0	0	11

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue L-1: User can speed up new deposit and unfairly inflate effective stake in PrimeLeaderboard deposit due to `_compactByTier` rounding down timestamps in favor of users [RESOLVED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/6

Summary

The effective stake in PrimeLeaderboard is calculated as `EffectiveStake = sum(deposit_amount * deposit_duration * multiplier)`. Each deposit is stored separately, but if the number of deposits reaches maximum set amount, they're compacted. The 2nd step, compact by tier, replaces all deposits within each tier by 1 deposit with sum deposit amount and weighted timestamp. The issue is that weighted timestamp is rounded down. This means that total duration of this single deposit is increased, so this rounding is in favor of user, slightly increasing his effective stake.

For example, if user has a deposit of 100e18 at timestamp = 200 and current timestamp = 300, he can deposit 0.9999e18 and it will be added to the deposit and due to rounding, timestamp of total deposit will remain at 200:

- `total_amount = 100.9999e18`
- `weighted_timestamp = (100e18 * 200 + 0.9999e18 * 300) / 100.9999e18 = 200 + 0.9999e18 * 100 / 100.9999e18 = round_down(200 + 0.99) = 200`

Just 100 deposit transactions will double his deposit of 100e18 while keeping timestamp (200) the same, inflating user's effective stake.

For another example, if user has a deposit of 2_600_000e18 29.9 days ago (30 days = 2_592_000 seconds), a deposit of 1e18 will be added without affecting timestamp due to rounding. This means that 2_600_000 deposits of 1e18 each will double his effective stake bypassing 30 days waiting period. If, say, 100 deposits are bundled into 1 transaction, 26K transactions are enough to do this. While the amount is quite large, it's still do-able, and provides a very significant and unfair boost to user's effective stake.

Root Cause

Rounding down weighted timestamp instead of rounding up: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/blob/51a5e98270ef8d548efa50e1219309f1e75c5ba2/venus-protocol/contracts/Tokens/Prime/PrimeLeaderboard.sol#L557

Internal Pre-conditions

- User has any deposit some time in the past (less than 30 days)

External Pre-conditions

None.

Attack Path

- Use multiple deposits to exceed total deposits limit with total amount 1 wei less than `previous deposit amount / time duration of previous deposit`
- After that all these deposits will be compacted within first tier and the amount added to previous deposit without moving the timestamp, effectively bypassing waiting this time and inflating the effective stake
- Repeat these deposits as many time as needed to increase deposit without moving its timestamp

Impact

Unfairly inflated user's effective stake. The longer the deposit duration (and the higher the impact), the smaller the amount which can be added, and the more transactions are required to perform the attack. Still, it's relatively easy to significantly inflate effective stake without having to risk large capital for a long time keeping it deposited and manipulate the leaderboard to get the prime v2 token just before the cutoff.

Mitigation

Round up weighted timestamp - this rounds the duration down to make rounding work against the user.

Discussion

Debugger022

Acknowledged as informational.

The rounding behaviour in `_compactDeposits` and `_compactByTier` is acknowledged. The attack, as described is not economically viable in practice:

Deposits enter the leaderboard exclusively via the `xvsUpdated` callback, which `XVSVault` invokes once per stake action. No multi-deposit batching interface exists at either the vault or the leaderboard, so each deposit requires a separate transaction. Combined with the per-cycle capital budget bounded by `S < A / age_seconds` and the 26 deposits

per compaction cycle required to retrigger compaction (`MAX_DEPOSITS_PER_USER = 30` minus the post-compaction residual of one entry per tier), the realistic transaction count to inflate effective stake meaningfully exceeds the value of the slot being protected even for whale-sized positions.

Additionally, the effective stake from the leaderboard governs only the binary top-500 eligibility cut-off for Prime token issuance. Reward distribution among Prime holders is driven by `_calculateScore` in `PrimeV2.sol`, which derives from XVS balance, capital, and the alpha curve – not from the leaderboard's effective stake. Any successful inflation moves a user across the eligibility boundary but does not change their per-epoch reward magnitude.

panprog

@Debugger022

Agree that the attack is likely not economically viable, that's why it's Low severity. Disagree with bundling multiple deposits - attacker-deployed contract can easily call `XVSVault.deposit` multple times in 1 transaction, and each call will trigger `xvsUpdated`.

The actual real-life scenario possible I see is if the user is slightly below the cutoff effective stake near the selection time and has some dated deposit (less than 30 days ago), so he can quickly inflate his effective stake a little to make it into top-500. This is unfair to the 500th user who will drop off and will lead to direct funds loss to this 500th user (inability to earn rewards he should be eligible for).

I still suggest fixing it as the fix is very simple (just ceil instead of round the weighted timestamp calculation): this is in-line with common recomendation of rounding against the user in most situations.

Debugger022

Confirmed: the deposits can be batched within a single transaction via a contract that calls `XVSVault.deposit` repeatedly, with each call triggering `xvsUpdated`. Thanks for clarifying that path.

The attack remains expensive and self-limiting – the per-cycle budget `S < A/age` shrinks as the deposit ages, so any meaningful inflation requires an impractical number of deposits, and effective stake only governs the binary top-500 eligibility cutoff, not reward magnitude. That said, the directional rounding bias is real, and the boundary scenario you describe – a user nudging just across the cutoff and displacing the legitimate 500th user – is a valid concern.

We've fixed it regardless e421dda, since the change is trivial: the amount-weighted timestamp is now ceiled in both merge paths (`_compactDeposits` pass-1 and `_compactByTier`), so the merged duration rounds down rather than in the user's favor – in line with the standard ”round against the user” guidance.

Issue L-2: `getPendingRewards` doesn't include removed markets where user still has accrued interest [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/7

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`getPendingRewards` and `getPendingRewardsStatic` functions iterate over all **active** markets, meaning that they don't include removed markets where user can still have accrued but not claimed interest. This can lead to incorrect UI display amount and complexities in claiming accrued interest in the removed markets.

Root Cause

There is no way to know all markets where user has non-0 accrued amount, only active markets are used: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/blob/51a5e98270ef8d548efa50e1219309f1e75c5ba2/venus-protocol/contracts/Tokens/Prime/PrimeV2.sol#L510-L520

Internal Pre-conditions

- User had accrued interest in the market which was later removed

External Pre-conditions

None.

Attack Path

Always happens by itself when pre-condition is met

Impact

Incorrect array of user's pending rewards (doesn't include rewards in removed markets)

Mitigation

Possible mitigations:

- Add a list of all markets which were ever active (never remove market from this list) and use this list for getting pending rewards
- Add per-user markets list where user has non-0 claim
- Fix off-chain by using keeper which will claim interest for all users upon market removal

Discussion

Debugger022

Acknowledged as informational.

User funds remain claimable in this scenario. `_claimInterest` explicitly handles removed markets – `interests[vToken][user].accrued` is preserved through `removeMarket` and can be withdrawn at any time via either the user-callable or the permissionless overload of `claimInterest`. This behavior is documented in the NatSpec at `PrimeV2.sol` and was introduced intentionally.

The `MarketRemoved` event (`PrimeV2.sol`) provides the off-chain signal needed to track removed markets. Indexers and frontends will consume this event to reconstruct the removed-market list and surface residual accrued amounts; a permissionless keeper can additionally call `claimInterest(removedMarket, user)` to settle balances on behalf of affected users. Maintaining an on-chain historical market list would expand `_allMarkets` iteration costs in every accrual and view call permanently to address a UI-only gap.

Issue L-3: `_ensureMaxLoops` uses the same fixed amount check for all loops while each loop can take very different amount of gas, especially in nested loops [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/8

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_ensureMaxLoops` is used to limit number of iterations in all the loops, using the same amount check for all loops. The issue is that different loops can consume very different amount of gas, meaning some loops are limited by too low amount for no reason. This limiter amount should be set using the worst (the most gas-heavy) loop. This is especially evident in nested loops, for example:

- `claimPrimeBatch` uses `_ensureMaxLoops` on the length of users array (who claim)
- for each user, `_initializeMarketsWithoutAccrual` is called, which also uses `_ensureMaxLoops` for markets length

Total number of iterations here is `O(N*N)`.

Obviously, the number of markets and the number of users might have very different limits. Most probably number of active markets is not very large, but number of users in the batch functions can be quite large. But the way it's setup now, both are forced to be the same amount.

Say, if it's safe to execute 1000 operations, then N should be set to sqrt(1000)=31 But if the actual market limit is 10, the users limit can be set to 1000/10 = 100 to allow processing of a lot more users per transaction.

Root Cause

Usage of single `_ensureMaxLoops` limit amount everywhere, for example: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/blob/51a5e98270ef8d548efa50e1219309f1e75c5ba2/venus-protocol/contracts/Tokens/Prime/PrimeV2.sol#L297 https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/blob/51a5e98270ef8d548efa50e1219309f1e75c5ba2/venus-protocol/contracts/Tokens/Prime/PrimeV2.sol#L911

Impact

Informational: suboptimal limits on the number of users updated in batch functions

Mitigation

Introduce multiple `_ensureMaxLoops` types and use different types for different loops.

Discussion

Debugger022

Acknowledged as informational. The single `loopsLimit` (currently 20) is applied uniformly across user-batch and market-iteration loops, which is conservative for the nested case but does not affect correctness or safety. The same single-limit pattern is used in PrimeV1 with no operational issues observed across its production lifetime, and at the current scale of Prime markets the resulting worst-case iteration count remains well within block gas limits.

No contract change planned. If batch throughput becomes a practical bottleneck in the future, the limit can be raised via `setMaxLoopsLimit` rather than introducing separate per-loop limits.

Issue L-4: Burning and minting Prime V2 tokens when switching between ”epochs” can introduce short time periods of unfair outsized (reduced) returns for low (high) score users [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/9

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

According to docs and comments, the intended process of minting/burning Prime V2 token for users is:

- Once every ”epoch” (30 days) off-chain admin bot calculates effective stake of each user in the leaderboard and sorts them by it
- Top 500 users are directly issued (minted) Prime V2, all previous Prime V2 holders' (which didn't make it into top 500 this epoch) Prime V2 tokens are burnt

Since it's most probably not possible to burn/mint all 500 users in 1 block, there will be a short transitional period when there're either too few or too many Prime V2 holders. This can cause outsized or reduced returns for some users.

For example:

- If the order is to first burn all previous holders, then mint all new holders, in case of large changes between epochs, right after burning all previous holders there might be just a few users with relatively small stake who will earn 100% of the distributed protocol revenue, which can be orders of mangitude higher than they actually deserve
- If the order is to first mint, then burn, then large holders will have their share of revenue significantly washed out by this temporary increase of users

Root Cause

Inability to burn/mint Prime V2 for all users during the switch between epochs

Impact

Informational: choose off-chain algorithm to burn/mint Prime V2 to ensure that after each transaction the revenue distribution is within normal bounds, i.e. prioritize burn, mint, burn, mint order and choose groups of users for burn and mint which are close in total stake and/or score.

Mitigation

Be aware of possible unfair revenue distribution and choose appropriate order of burning and minting.

Discussion

Debugger022

No funds are lost in this scenario. `_burnWithoutAccrual` snapshots each departing holder's accrued interest before subtracting their score from `sumOfMembersScore`, and `_claimInterest` allows them to withdraw it after burn. Only the distribution of new PLP income arriving during the transition window is affected.

The PrimeLiquidityProvider streams rewards continuously, accruing `deltaBlocks * distributionSpeed` per call, so the realised skew during a typical transition window is bounded by the configured per-block distribution speed and the duration of the gap.

This is handled operationally. The keeper bot interleaves burns and mints in score-balanced groups so that `sumOfMembersScore` stays close to its steady-state value throughout the transition, and all transition transactions are dispatched consecutively from the same operator to keep the wall-clock window minimal.

Acknowledged as informational; no contract change planned.

fred-venus

I'm okay to mark this as a ”ACK”, as the reward is distributed per block, and we can conservatively say we will finish the Prime token reallocation in no more than 5 minutes, 5 mins over 1 month is ~0.0116% which can be safely ignored

Issue L-5: Income accrued while there are no users with capital in the market is permanently lost [RESOLVED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/10

Summary

When `accrueInterest(market)` is called, it distributes accrued income to all users with capital in the market:

```
uint256 totalAccruedInPLP = _primeLiquidityProvider.tokenAmountAccrued(underlying);
uint256 unreleasedPLPAccruedInterest = totalAccruedInPLP -
↪  unreleasedPLPIncome[underlying];
uint256 distributionIncome = unreleasedPLPAccruedInterest;

if (distributionIncome == 0) {
    return;
}

unreleasedPLPIncome[underlying] = totalAccruedInPLP;

uint256 delta;
if (market.sumOfMembersScore != 0) {
    delta = ((distributionIncome * EXP_SCALE) / market.sumOfMembersScore);
}

market.rewardIndex += delta;
```

The issues is that in the special edge case when there are no users with capital in the market (`sumOfMemberScores == 0`), the income is marked as distributed (`unreleasedPLPIncome[underlying]` is set to new updated total accrued amount), but `market.rewardIndex` is not increased.

This means that when the first user's score in the market becomes positive, he starts with 0 base and this income marked as distributed is never sent anywhere.

Root Cause

`unreleasedPLPIncome` advanced even if it's not actually distributed/added to users: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/blob/51a5e98270ef8d548efa50e1219309f1e75c5ba2/venus-protocol/contracts/Tokens/Prime/PrimeV2.sol#L469-L474

Internal Pre-conditions

- Market without users with non-0 score (for example, a newly-added market)
- `accrueInterest` is called for the market when the market has some revenue to distribute

External Pre-conditions

None.

Attack Path

Always happens by itself when pre-conditions are met

Impact

The income distributed in the market during time period when internal pre-conditions are met is permanently lost

Mitigation

Either:

- Send such income to some admin-controlled address for further re-distribution; OR
- Only advance `unreleasedPLPIncome` if `sumOfMembersScore != 0`, meaning that the first user to have non-0 score will get all this undistributed revenue (probably less fair option)

Discussion

Debugger022

Fixed 61625b1

fred-venus

i prefer we keep it as-is, the rationale behind this:

prime reward is actually to incentivize people to have XVS staking as well as prime supporting markets interaction. It is under the assumption that at least the rewarded user has XVS staking and otop of this, they need to have at least some supply in corresponding markets.

Basically, if there are literally no people contributing to our protocol, it doesn't make sense for us to distribute the reward or carry forward the reward for the future user. We only reward users' behavior when they contribute.

As for the fund, its not lost, it will remain to sit in the contract and wait for the community voted vip to get it reallocated

Debugger022

Reverted: 2f809a8

panprog

@fred-venus

> As for the fund, its not lost, it will remain to sit in the contract and wait for the community voted vip to get it reallocated

Can you give more details on how exactly it will be reallocated? I currently see no way to reallocate these funds: these funds are either in the liquidity provider (owed to PrimeV2) or in PrimeV2 depending on liquidity available to pay out to users. Either way there are no functions to forcibly send these funds anywhere. Users can claim what is owed to them, but admin can't claim excess funds distributed to PrimeV2 but not recorded to any user.

The mitigation is to either send funds to future users or to add a function to reallocate these funds, i.e. send to some admin-controlled address or accumulate them in the contract via some additional function. Unless I'm missing some way this is already available.

fred-venus

Hey @panprog , thx for reminding this, you are right, i didn't notice PLP has a `sweepToken` but PrimeV2 doesn't, so once the fund is released it got locked forever, then how about

```
mapping(address => uint256) public undistributedReward;

function accrueInterest(address vToken) public {
    ...

    if (market.sumOfMembersScore != 0) {
        market.rewardIndex += (distributionIncome * EXP_SCALE) /
        ↪  market.sumOfMembersScore;
    } else {
        undistributedReward[underlying] += distributionIncome;   // ← jot down
    }
}

function sweep() ... // ACM controlled
```

cc @Debugger022 , wdyt ?

Debugger022

@fred-venus your ledger approach works – keeps bookkeeping on-chain, bounds the sweep amount so we can't accidentally drain user-owed funds. Two refinements:

1. `accrueInterest(vToken)` first inside sweep, so any pending PLP delta lands in the right bucket (rewardIndex if score > 0, undistributedReward otherwise) before we read the ledger.
2. If Prime's `balance < undistributedReward[underlying]`, call `releaseFunds` to pull from PLP – mirrors the existing `_claimInterest` pattern (delete `unreleasedPLPIncome[underlying]` then `releaseFunds`, both sides reset atomically).

```
function sweepUndistributed(address vToken, address to) external {
    _checkAccessAllowed("sweepUndistributed(address,address)");
    address underlying = _getUnderlying(vToken);

    accrueInterest(vToken);

    uint256 amount = undistributedReward[underlying];
    if (amount == 0) return;

    uint256 available = IERC20Upgradeable(underlying).balanceOf(address(this));
    if (amount > available) {
        delete unreleasedPLPIncome[underlying];
        IPrimeLiquidityProvider(primeLiquidityProvider).releaseFunds(underlying);
    }

    delete undistributedReward[underlying];
    IERC20Upgradeable(underlying).safeTransfer(to, amount);
    emit UndistributedSwept(underlying, to, amount);
}
```

Alt option if we want to stay minimal – generic ACM sweep mirroring `PLP.sweepToken`:

```
function sweep(address token, address to, uint256 amount) external {
    _checkAccessAllowed("sweep(address,address,uint256)");
    if (to == address(0)) revert InvalidAddress();
    uint256 balance = IERC20Upgradeable(token).balanceOf(address(this));
    if (amount > balance) revert InsufficientBalance(amount, balance);
    IERC20Upgradeable(token).safeTransfer(to, amount);
    emit SweepToken(token, to, amount);
}
```

Minimal, also recovers accidental transfers, same trust model Venus accepts for PLP. Generic sweep is fine if PLP parity is preferred. Your call.

Storage gap on PrimeV2StorageV1 is 41 slots, mapping uses 1, no layout issue either way.

Debugger022

We've added a ledger + ACM-gated sweep in b4fca63

- `mapping(address => uint256) public undistributedReward;` records the per-underlying slice every time `accrueInterest` runs with `sumOfMembersScore == 0`.

- `sweepUndistributed(address vToken, address to)` (ACM-gated) flushes any pending PLP delta via `accrueInterest` first, pulls funds from `PrimeLiquidityProvider` via `releaseFunds` if Prime's local balance is short (mirroring the `_claimInterest` pattern that deletes `unreleasedPLPIncome[underlying]` and resets both sides atomically), then transfers exactly `undistributedReward[underlying]` to the recipient and emits `UndistributedSwept`.

The sweep amount is bounded on-chain by the ledger, so there is no risk of over-sweeping into user-owed balances. Recovery is governance-driven (Normal Timelock) via a community-approved VIP.

panprog

@Debugger022 The `sweepUndistributed` differs from `_claimInterest` in handling lack of liquidity situation: if there are not enough funds even after releasing funds from liquidity provider, `_claimInterest` sends everything available and keeps the rest as accrued to user, while `sweepUndistributed` simply reverts. Is there any specific reason for this? Not an issue, I suppose if there is not enough liquidity, admin shouldn't sweep these funds anyway. Still if there is small liquidity shortage, where all liquidity still covers all users funds, but only part of undistributed rewards are covered, these funds will still be stuck as all attempts to sweep them will revert (instead of sending whatever is available).

Debugger022

@panprog By design – `sweepUndistributed` is an admin/governance action, not a user-facing flow, so reverting on a liquidity shortage is acceptable: it surfaces the shortage to the caller and prevents partially-completed sweeps from fragmenting the residual across multiple admin transactions. No funds are at risk – the revert rolls back the `delete undistributedReward[underlying]`, so the full amount remains tracked and the sweep can be retried once liquidity is sufficient (either after additional PLP accrual or a top-up).

Issue L-6: User score recalculation can be triggered by any user [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/11

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

Any user can trigger score re-calculation for any other user via these functions:

- `accrueInterestAndUpdateScore(user, market)`
- `accrueInterestAndUpdateScore(user)`
- `updateScores(user)` (limited - only when score update is pending)

These functions are called infrequently for most users, so user scores are mostly updated when user balance changes or there is significant underlying price change.

However, malicious users can monitor underlying price and any time it drops - update scores for all the other users, keeping their own score unchanged (calculated when underlying price was higher). If the underlying price increases - they can update only their own score. This unfairly increases their share of the revenue distribution.

Root Cause

No access control for functions which update user score: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/blob/51a5e98270ef8d548efa50e1219309f1e75c5ba2/venus-protocol/contracts/Tokens/Prime/PrimeV2.sol#L485 https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/blob/51a5e98270ef8d548efa50e1219309f1e75c5ba2/venus-protocol/contracts/Tokens/Prime/PrimeV2.sol#L496 https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/blob/51a5e98270ef8d548efa50e1219309f1e75c5ba2/venus-protocol/contracts/Tokens/Prime/PrimeV2.sol#L565

Internal Pre-conditions

None.

External Pre-conditions

Underlying price drops.

Attack Path

- Call any of the 2 (3) functions on all the other users to reduce their score, while keeping your own score unchanged (higher than current price implied)

Impact

Unfairly inflates attacker's revenue share.

Mitigation

Add access control to these functions so that all users are updated fairly by the keeper bot.

Note: in the fix review PR this is partially done, but only for the "accrueInterestAndUpdateScore(user)' function, not the other 2.

Discussion

Debugger022

Acknowledged.

1. **V1 parity.** `accrueInterestAndUpdateScore(address,address)` and `updateScores(address[])` are public in PrimeV1 with the same semantics. Live since launch, no exploitation observed.
2. **Issuance is permissioned in V2.** `issue` / `issueBatch` / `burn` / `burnBatch` are ACM-gated. The permissionless `claimPrime` / `claimPrimeBatch` paths require `getEffectiveStake(user) >= mintThreshold`, so the Prime holder set is determined by the leaderboard, not by attacker action.
3. **Score is mostly XVS-bound.** `_calculateScore` caps on `(xvsBalance, capital)` via the alpha curve. For stake-ranked holders the cap is dominated by XVS holdings, making the score insensitive to short oracle dips.
4. **Attacker self-update is forced.** Any supply/borrow/redeem/repay/transfer on a boosted vToken triggers Comptroller hooks that re-snapshot the attacker's own score. Holding a stale high score requires complete inactivity, which kills the economic upside.
5. **Round bounds `updateScores`.** `isScoreUpdated[nextScoreUpdateRoundId][user]` guarantees one update per user per round; any asymmetry resolves at round close.
6. **Targeted variant already gated.** `accrueInterestAndUpdateScore(address)` is restricted to `primeLeaderboard` (`PrimeV2.sol` L510) to block forced refreshes on stake transitions. Remaining two entry points kept permissionless for keeper fault tolerance and V1 compatibility.

Issue L-7: Compacting deposits unfairly inflates the top deposit duration in pass 1, inflating user score if max multiplier duration is ever increased [RESOLVED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/12

Summary

When deposits are compacted in pass 1, the earliest timestamp is used for merged deposit. While the actual timestamp doesn't matter at the time it's compacted, if the max multiplier duration ever changes, the user's effective stake can become inflated.

Root Cause

Usage of the earliest timestamp from all deposits: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/blob/51a5e98270ef8d548efa50e1219309f1e75c5ba2/venus-protocol/contracts/Tokens/Prime/PrimeLeaderboard.sol#L500

Internal Pre-conditions

- Admin adds new tier with longer duration or changes existing top tier by increasing its duration

External Pre-conditions

None.

Attack Path

Always happens by itself

Impact

If user had multiple deposits, some of which were in top tier but below the new max duration, and some of which above new max duration, all such deposits will be merged into single deposit at max duration, unfairly inflating user effective stake.

Mitigation

Use weighted timestamp instead of the earliest timestamp as in pass 2. Note: already fixed in fix PR.

Discussion

Debugger022

Already fixed: 649c015

Issue L-8: Stale Prime holders are never burned when effective stake drops below the mint threshold [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/13

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`claimPrime` blocks a fresh mint when the user's effective stake is below `mintThreshold`, but there is no symmetric path that revokes an existing Prime token once the holder's stake decays under the same threshold. Eligibility is checked only at mint time, so ineligible holders keep accruing boosted rewards indefinitely.

Vulnerability Details

`PrimeV2.claimPrime` reads the user's effective stake from `PrimeLeaderboard` and reverts when that score is below the configured `mintThreshold`.

```
uint256 score = IPrimeLeaderboard(leaderboard).getEffectiveStake(user);
if (score < threshold) revert EligibilityBelowThreshold(user, score, threshold);

_mint(user);
_initializeMarkets(user);
```

The leaderboard contract is the source of truth for staking activity and is the only component that observes withdrawals through `xvsUpdated`. When a withdrawal lands, `PrimeLeaderboard.xvsUpdated` records the withdrawal and forwards a call to `accrueInterestAndUpdateScore(user)` on `PrimeV2`. That call only refreshes per-market scores and reward indexes. It does not check `mintThreshold` and it never invokes `burn`.

```
if (primeV2 != address(0)) {
    IPrimeV2(primeV2).accrueInterestAndUpdateScore(user);
}
```

`PrimeV2` only exposes `burn` and `burnBatch`, both gated by ACM and intended for manual operator use. There is no permissionless or automatic path that compares `getEffectiveStake(user)` against `mintThreshold` for existing holders. Because the threshold is checked exclusively at mint time, a user who minted while eligible and then withdrew the majority of their XVS keeps their Prime token. Their per-market score is recomputed downward by `_updateScore`, but the Prime token itself is never revoked. They continue to share in `sumOfMembersScore` and continue to receive rewards from `accrueInterest`.

Impact

Holders whose effective stake has decayed below `mintThreshold` keep collecting boosted yield they would not be allowed to start collecting today. New eligible stakers cannot displace them either, because `tokenLimit` is global and Prime issuance does not evict the lowest ranked holder. Honest stakers above the threshold can be locked out of minting once `totalTokens` reaches `tokenLimit`, while the seats are held by addresses that no longer meet the eligibility bar. The economic loss accrues continuously through every `accrueInterest` cycle in every market the stale holder is enrolled in.

Recommendation

The leaderboard already calls into `PrimeV2` from `xvsUpdated`. Extend that path so that when a Prime holder's effective stake drops under `mintThreshold`, the leaderboard triggers a burn on `PrimeV2`. Alternatively, expose a permissionless `revokeIneligible(address user)` on `PrimeV2` that re-checks the leaderboard score against `mintThreshold` and burns the token if it is below. This restores symmetry between mint and burn eligibility and prevents indefinite stale holders.

Discussion

Debugger022

The ”indefinite boosted yield” framing does not hold: a holder's per-market score is recomputed from their current XVS balance on every stake change (`xvsUpdated → accrueInterestAndUpdateScore → _updateScore`), and `Scores._calculateScore` returns 0 when `xvs == 0`. A holder who withdraws all XVS therefore earns nothing immediately even while still holding the token, and a partial withdrawal scales their reward share down proportionally via `xvŝ` . There is no path where a decayed holder keeps collecting their original reward magnitude.

The residual effect is slot occupancy – a holder below `mintThreshold` but with a non-zero score keeps a `tokenLimit` seat. This is handled by design: eviction is an epoch-level keeper operation that recomputes the ranking and burns dropouts via `burnBatch` each cycle, so any squat is bounded by the epoch cadence while the squatting holder earns only score-proportional (reduced) rewards in the interim.

Issue L-9: updateScores re-accrues every batch and front-loads index drift onto users updated first [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/14

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`updateScores` calls `_accrueAllMarkets` on every keeper batch. Each accrual call advances `rewardIndex` against the current `sumOfMembersScore`, which still includes the old scores of users that have not yet been updated this round. Users whose scores rise in the round are paid out on the old denominator until their turn comes, and users whose scores fall are still credited as if they were oversized. The order in which the keeper visits users changes who wins and who loses.

Vulnerability Details

The function is intentionally split into many small calls so a keeper can update all holders across multiple transactions inside a single round.

```
function updateScores(address[] calldata users) external {
    if (pendingScoreUpdates == 0) revert NoScoreUpdatesRequired();

    uint256 usersLength = users.length;
    _ensureMaxLoops(usersLength);

    _accrueAllMarkets();

    for (uint256 i; i < usersLength; ) {
        address user = users[i];

        if (!isPrimeHolder[user]) {
            unchecked { ++i; }
            continue;
        }

        if (isScoreUpdated[nextScoreUpdateRoundId][user]) {
            unchecked { ++i; }
            continue;
        }

        _accrueInterestAndUpdateScoreWithoutAccrual(user);
        isScoreUpdated[nextScoreUpdateRoundId][user] = true;
```

```
        unchecked {
            --pendingScoreUpdates;
            ++i;
        }

        emit UserScoreUpdated(user);
    }
}
```

`_accrueAllMarkets` advances each market's `rewardIndex` by `distributionIncome / sumOfMembersScore`. During a score update round, `sumOfMembersScore` is a mix of fresh scores (for users already processed in earlier batches) and stale scores (for users still queued). Between two keeper transactions inside the same round, the protocol keeps minting `rewardIndex` against this mixed denominator.

`_accrueInterestAndUpdateScoreWithoutAccrual` then calls `_executeBoostWithoutAccrual` per user, which uses the current per-market `rewardIndex` as the cutoff and credits the delta to that user's `accrued`. A user processed earlier in the round has their old (often inflated or deflated) score multiplied by the index delta that was generated using the stale `sumOfMembersScore`. A user processed later picks up a different index delta computed against a mix that has already started shifting.

The accrual could be hoisted out of `updateScores` entirely. The trigger that started the round (`addMarket`, `removeMarket`, `updateMultipliers`, `updateAlpha`) calls `_queueScoreUpdates`, but none of these snapshot the index, so accrual keeps running during the multi-tx window.

Impact

Reward distribution inside a score update round is path-dependent on keeper ordering. A user whose score is about to be revised downward benefits if the keeper processes them last, because the index delta accrued during the round is multiplied against their still-inflated stored score. A user whose score is about to be revised upward is harmed if processed first, because the round's index drift is split against everybody else's stale `sumOfMembersScore` rather than their fresher larger score. The size of the skew scales with `distributionIncome` arriving from `PrimeLiquidityProvider` during the round and with the number of batches the keeper splits the round into. There is no direct loss-of-funds path, but the keeper can deterministically bias who collects more by choosing the visit order.

Recommendation

Snapshot index state at the start of the round so that all users in the round are credited against the same `rewardIndex` boundary, regardless of the batch they fell into. A clean way to do this is to call `_accrueAllMarkets` once when `_queueScoreUpdates` is triggered, then drop the per-batch `_accrueAllMarkets` call inside `updateScores`. Resume accrual

only after `pendingScoreUpdates` returns to zero. This removes keeper ordering as a degree of freedom and makes the round's reward share deterministic.

Discussion

Debugger022

Acknowledged as informational. No funds are lost – income accrued across a round's batches is fully distributed and each holder is updated exactly once per round (`isScoreUpdated[nextScoreUpdateRoundId][user]`); only the allocation of mid-round income shifts with visit order.

Rounds can only be opened by ACM-gated governance operations (`addMarket`, `removeMarket`, `updateAlpha`, `updateMultipliers`), so no external actor can trigger one, and `updateMultipliers` settles that market's pending income via `accrueInterest` before queuing the round. The exploitable surface is thus limited to income streaming from the `PrimeLiquidityProvider` during an already-open round – bounded by the per-block distribution speed and the few blocks the keeper takes to close the round, on the order of cents per round, and only when the caller's own score is decreasing that round.

This mirrors PrimeV1 (same permissionless `updateScores`, same round counters, same mid-round accrual via `_executeBoost/accrueInterest`), live since 2024 with no observed exploitation. The suggested mitigation (snapshot the index at round start, freeze accrual until close) only trades the ordering bias for another – mid-round income would be distributed at end-of-round scores instead of the scores active when it accrued – while adding a new frozen-accrual state. Given conserved funds, governance-only triggering, cents-scale magnitude, and V1 parity, no contract change is planned.

Issue L-10: removeMarket is blocked whenever any Prime holder has a non zero score in that market [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/15

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`removeMarket` reverts unless `sumOfMembersScore` for the target market is zero. Because the only way to zero that figure is to burn (or reduce to zero score) every active Prime holder enrolled in the market, governance cannot actually retire a market without first wiping the entire Prime cohort that touched it. `_claimInterest` is already designed to pay residual rewards after removal, so the guard is over-restrictive relative to the protocol's stated intent.

Vulnerability Details

The guard sits at the top of `removeMarket`.

```
function removeMarket(address market) external {
    _checkAccessAllowed("removeMarket(address)");

    if (!markets[market].exists) revert MarketNotSupported();
    if (markets[market].sumOfMembersScore > 0) revert MarketHasActiveMembers();
```

`sumOfMembersScore` is set inside `_initializeMarketsWithoutAccrual` on every new Prime mint and updated by `_updateScore` whenever a user's per-market score changes. It is only reduced to zero by `_burnWithoutAccrual`, which deletes the user's per-market `score` and subtracts it from the running sum.

```
markets[market].sumOfMembersScore = markets[market].sumOfMembersScore -
↪  interests[market][user].score;
delete interests[market][user].score;
delete interests[market][user].rewardIndex;
```

So `sumOfMembersScore > 0` whenever there is at least one Prime holder with a positive per-market score. With a live Prime cohort this is the default state for every productive market. To meet the `MarketHasActiveMembers` precondition, governance must call `burn` or `burnBatch` for every such holder first, which destroys their Prime status across every other market too. The contract already accepts this is wasteful in the post-removal path: `_claimInterest` explicitly checks `bool marketExists = markets[vToken].exists` and falls

through to `amount += interests[vToken][user].accrued` so users can still pull residual interest after a market is removed.

```
function _claimInterest(address vToken, address user) internal returns (uint256) {
    bool marketExists = markets[vToken].exists;

    uint256 amount;
    if (marketExists && isPrimeHolder[user]) {
        accrueInterest(vToken);
        amount = _interestAccrued(vToken, user);
        interests[vToken][user].rewardIndex = markets[vToken].rewardIndex;
    }
    amount += interests[vToken][user].accrued;

    if (!marketExists && amount == 0) revert MarketNotSupported();
```

The two halves of the contract disagree. `removeMarket` says the market cannot be removed unless nobody is in it. `_claimInterest` says removed markets are expected to have residual member balances. The guard makes the residual-claim code path unreachable in the only sequence that is supposed to reach it.

Impact

Governance cannot retire an obsolete market without administratively burning every Prime holder that participated in it. That collateral damage is not local to the removed market: `_burnWithoutAccrual` deletes the user's score across every market, so the user has to be re-issued from scratch and goes through `_initializeMarkets` again, restarting their per-market reward indexes. In practice this turns `removeMarket` into a footgun that nobody can use, and the residual-claim machinery in `_claimInterest` is dead code.

Recommendation

Drop the `sumOfMembersScore > 0` revert and rely on the existing residual-claim path in `_claimInterest`. Before removal, accrue interest one final time and snapshot the market's final `rewardIndex` so per-user accrued amounts can be settled from `.accrued`. State that should be cleared (per-user `score` and `rewardIndex` in the removed market) should be migrated as part of the removal flow or the next user interaction, not by forcing the cohort to be burned first.

Discussion

Debugger022

Acknowledged as informational. The guard is an intentional safety invariant, and the ”dead code” framing does not hold.

A user's per-market score is $xv\hat{s} \cdot capital\tilde{}(1-)$, and `capital` derives from their supply/borrow in that specific market, so `score == 0` whenever a holder has no position there. `sumOfMembersScore == 0` therefore means no Prime holder is still being boosted in the market – the guard only blocks removal while someone holds an active boosted position in it.

Crucially, `score == 0` does not imply `accrued == 0`. When a holder exits their position, the Comptroller hook calls `_executeBoost`, which settles pending interest into `interests[market][user].accrued` *before* `_updateScore` zeroes the score. After all holders exit, `removeMarket` is permitted while residual `accrued` persists, and `_claimInterest` pays it out via the `marketExists == false` branch (`amount += interests[vToken][user].accrued`). The residual-claim path is reachable, not dead – the removal guard keys on `score`, the residual claim keys on `accrued`, and the two are independent.

Dropping the guard would allow a market to be removed while holders still have non-zero scores, stranding those scores in storage and breaking accrual accounting (`accrueInterest` reverts once `market.exists` is false). `removeMarket` is a rare governance operation, and the wind-down is driven Comptroller-side (caps to zero, forced exits) so positions clear before removal.

Issue L-11: _calculateScore uses exchangeRateStored which can be stale relative to the user's actual supply position [ACKNOWLEDGED]

Source: https://github.com/sherlock-audit/2026-05-venus-protocol-may-18th-2026/issues/16

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_calculateScore` converts a user's vToken balance to underlying using `vToken.exchangeRateStored()`. That function reads the last-checkpointed exchange rate without accruing interest in the vToken first. Whenever the vToken has not been touched since the last interest accrual, the supply value used to compute the user's Prime score under-represents the user's real underlying position by exactly the unaccrued interest delta.

Vulnerability Details

The score calculation:

```
function _calculateScore(address market, address user) internal returns (uint256) {
    uint256 xvsBalanceForScore = _xvsBalanceOfUser(user);

    IVToken vToken = IVToken(market);
    uint256 borrow = vToken.borrowBalanceStored(user);
    uint256 exchangeRate = vToken.exchangeRateStored();
    uint256 balanceOfAccount = vToken.balanceOf(user);
    uint256 supply = (exchangeRate * balanceOfAccount) / EXP_SCALE;

    oracle.updateAssetPrice(xvsVaultRewardToken);
    oracle.updatePrice(market);

    (uint256 capital, , ) = _capitalForScore(xvsBalanceForScore, borrow, supply,
    ↪  market);
```

`exchangeRateStored` is the Venus pattern for "last computed rate, without accruing." It reflects the rate as of the last block in which `accrueInterest` was called on the vToken itself. The function takes pains to refresh prices via `oracle.updateAssetPrice` and `oracle.updatePrice(market)` so that the dollar-cap computation in `_capitalForScore` uses fresh prices, but the underlying token balance fed into that computation is still based on a stale exchange rate. The same applies to `borrowBalanceStored`, which does not accrue the borrow side either.

`_calculateScore` is invoked from `_initializeMarketsWithoutAccrual` and `_updateScore`, which are the paths that set `interests[market][user].score`. In both paths the contract has just called `_accrueAllMarkets` to advance Prime's own `rewardIndex`, but that accrual is internal to the Prime contract and does not propagate into the vToken's `accrueInterest`. As a result, the supply leg of the user's capital is whatever the vToken last checkpointed, even if many blocks have passed since.

The skew is one-sided. Supply value grows over time (interest accrues to suppliers), so the stored exchange rate is always behind the fresh exchange rate. Borrow balance also grows over time, and `borrowBalanceStored` is also behind. The two errors do not cancel: they get added together inside `_capitalForScore` as `cappedSupply + cappedBorrow`. So `capital` is always under-stated by the unaccrued interest delta on both sides.

Impact

A user's Prime score is under-counted by the size of the vToken's unaccrued interest delta whenever `_calculateScore` runs against a vToken that has not been touched recently. Because every Prime score update flows into `sumOfMembersScore`, this also drags down the denominator used by `accrueInterest` when distributing `PrimeLiquidityProvider` income. A user that triggers their score recalculation immediately after a fresh vToken accrual ends up with a strictly higher score than one whose score was recalculated against stale state, even though their underlying positions are identical. Sophisticated callers can monitor vToken block-level activity and time their own `accrueInterestAndUpdateScore` invocations to land right after a vToken `accrueInterest`, while passive users absorb the staleness. The differential is bounded by the time since the vToken's last accrual and the vToken's borrow rate, so on actively used markets it is small, while on quiet markets with infrequent accrual it can be material.

Recommendation

Call `vToken.accrueInterest()` once on entry to `_calculateScore` (or on entry to the higher-level paths `_accrueInterestAndUpdateScoreWithoutAccrual` and `_initializeMarketsWithoutAccrual`) before reading `exchangeRateStored` and `borrowBalanceStored`. After the accrual, the stored values match the current block and the score reflects the user's real supply and borrow position. The fresh-price calls below the exchange-rate read already accept that the function is not view, so adding the accrual call carries no API change.

Discussion

Debugger022

Acknowledged as informational.

The dominant update path is fresh by construction: every position change accrues the vToken before the Prime hook fires (`mintInternal/redeemInternal/borrowInternal/repay`

BorrowInternal/liquidateBorrowInternal all call accrueInterest() before *Fresh → comptroller.*Allowed → prime.accrueInterestAndUpdateScore). So a user's score is computed against freshly-accrued state whenever their supply or borrow changes.

Staleness is confined to non-hook updates (keeper updateScores, direct accrueInterestAndUpdateScore, xvsUpdated, and transfer), where the lag is one-sided and bounded by (time since the vToken's last accrual) × borrow rate – negligible on active markets, material only on quiet ones. And because _capitalForScore caps both legs at the XVS-derived USD cap, XVS-bound holders (the typical Prime profile) are unaffected entirely – their capped capital is independent of the stale exchangeRateStored/borrowBalanceStored. Only capital-bound users (small XVS relative to position) can be touched, by the accrual delta.

There is no protocol loss – understated capital lowers the user's score and the sumOfMembersScore denominator together – and it self-corrects on any subsequent position change, or via vToken.accrueInterest() + accrueInterestAndUpdateScore.

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.